

Composición de Componentes

Objetivos

- Entender por qué React favorece composición sobre herencia
 - Aplicar patrones de composición en componentes
 - Crear componentes genéricos y reutilizables
 - Usar Higher-Order Components (HOCs) cuando sea apropiado
-

Contenido

1. ¿Por qué Composición en React?

React adopta un modelo de **composición** en lugar de herencia de clases. Esto significa que construimos componentes combinando otros componentes, no extendiendo clases base.

```
// ❌ NO en React - Herencia (evitar)
class Button extends BaseComponent {
  render() {
    return <button>{this.props.text}</button>;
  }
}

// ✅ SÍ en React - Composición
// QUÉ: Componente base que acepta children y variant como props
// PARA: Crear un botón flexible que puede contener cualquier contenido
// IMPACTO: Un solo componente sirve para múltiples casos de uso
const Button: React.FC<ButtonProps> = ({ children, variant }) => {
  return <button className={`btn btn-${variant}`}>{children}</button>;
};

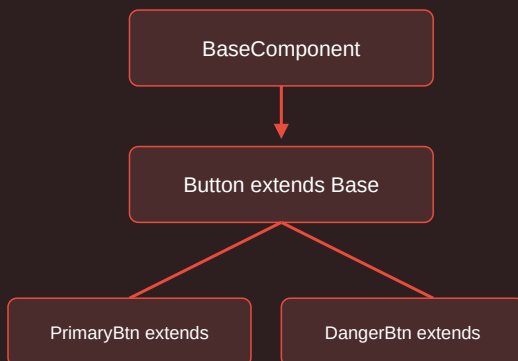
// QUÉ: Componentes especializados que envuelven al componente base
// PARA: Crear variantes pre-configuradas sin duplicar código
// IMPACTO: API simple para el usuario, mantenimiento centralizado
const PrimaryButton: React.FC<{ children: React.ReactNode }> = ({
  children,
}) => <Button variant="primary">{children}</Button>;

const DangerButton: React.FC<{ children: React.ReactNode }> = ({
  children,
}) => <Button variant="danger">{children}</Button>;
```

2. Beneficios de la Composición

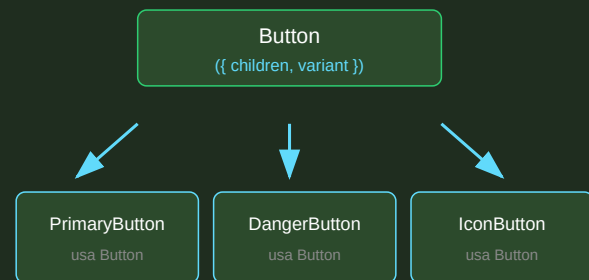
Composición vs Herencia en React

❌ Herencia (Evitar)



- Jerarquía rígida
- Alto acoplamiento
- Difícil de modificar
- Hereda todo o nada

✅ Composición (Preferir)



- Componentes independientes
 - Bajo acoplamiento
- Fácil de modificar y testear
- Combina solo lo necesario
- Reutilización flexible

```
<Button variant="primary">{children}</Button>
```

2.1 Flexibilidad

La composición permite combinar componentes de formas que no anticipamos:

```
// QUÉ: Interface que define props con children genérico
// PARA: Permitir que Card acepte cualquier contenido JSX
// IMPACTO: Máxima flexibilidad - el padre decide qué renderizar
interface CardProps {
  children: React.ReactNode;
  className?: string;
}

// QUÉ: Componente contenedor que solo proporciona estructura visual
// PARA: Encapsular estilos sin imponer estructura de contenido
// IMPACTO: Reutilizable en contextos completamente diferentes
const Card: React.FC<CardProps> = ({ children, className = '' }) => (
  <div className={`card ${className}`}>{children}</div>
);

// Múltiples formas de usarlo
const UserProfile: React.FC<{ user: User }> = ({ user }) => (
  <Card>
    <img
      src={user.avatar}
      alt={user.name}
    />
    <h2>{user.name}</h2>
    <p>{user.bio}</p>
  </Card>
);

const ProductCard: React.FC<{ product: Product }> = ({ product }) => (
```

```

<Card className="product-card">
  <img
    src={product.image}
    alt={product.name}
  />
  <h3>{product.name}</h3>
  <span className="price">${product.price}</span>
</Card>
);

const NotificationCard: React.FC< { message: string } > = ({ message }) => (
  <Card className="notification">
    <p>{message}</p>
  </Card>
);

```

2.2 Reutilización

Componentes pequeños y enfocados son más fáciles de reutilizar:

```

// QUÉ: Componentes atómicos - la unidad más pequeña de UI
// PARA: Crear piezas reutilizables que hacen una sola cosa bien
// IMPACTO: Se pueden combinar de infinitas formas sin modificar el código
const Avatar: React.FC<{
  src: string;
  alt: string;
  size?: 'sm' | 'md' | 'lg';
}> = ({ src, alt, size = 'md' }) => (
  <img
    src={src}
    alt={alt}
    className={`avatar avatar-${size}`}
  />
);

const Badge: React.FC<{ children: React.ReactNode; color?: string }> = ({
  children,
  color = 'blue',
}) => <span className={`badge badge-${color}`}>{children}</span>;

const Text: React.FC<{
  children: React.ReactNode;
  variant?: 'body' | 'caption';
}> = ({ children, variant = 'body' }) => (
  <p className={`text-${variant}`}>{children}</p>
);

// QUÉ: Componente compuesto que usa los atómicos
// PARA: Crear UI más compleja combinando piezas simples
// IMPACTO: Código declarativo, fácil de leer y mantener
const UserBadge: React.FC<{ user: User }> = ({ user }) => (
  <div className="user-badge">
    <Avatar
      src={user.avatar}
      alt={user.name}
      size="sm"
    />
  </div>
);

```

```

    />
    <Text variant="body">{user.name}</Text>
    {user.isAdmin && <Badge color="gold">Admin</Badge>}
  </div>
);

```

2.3 Testabilidad

Componentes compuestos son más fáciles de testear:

```

// Fácil de testear individualmente
describe('Avatar', () => {
  it('renders with correct size class', () => {
    render(
      <Avatar
        src="/test.jpg"
        alt="Test"
        size="lg"
      />,
    );
    expect(screen.getByRole('img')).toHaveClass('avatar-lg');
  });
});

describe('Badge', () => {
  it('renders children correctly', () => {
    render(<Badge color="green">Active</Badge>);
    expect(screen.getByText('Active')).toBeInTheDocument();
  });
});

// Fácil de testear la composición
describe('UserBadge', () => {
  it('shows admin badge for admin users', () => {
    const adminUser = { name: 'Admin', avatar: '/admin.jpg', isAdmin: true };
    render(<UserBadge user={adminUser} />);
    expect(screen.getByText('Admin')).toBeInTheDocument();
  });
});

```

3. Patrones de Composición

3.1 Contenedor y Presentación

Separar lógica de presentación:

```

// Componente de presentación - solo UI
interface UserListViewProps {
  users: User[];
  onSelectUser: (user: User) => void;
  selectedId?: string;
}

const UserListView: React.FC<UserListViewProps> = ({
  users,
  onSelectUser,

```

```

        selectedId,
    }) => (
        <ul className="user-list">
            {users.map((user) => (
                <li
                    key={user.id}
                    className={user.id === selectedId ? 'selected' : ''}
                    onClick={() => onSelectUser(user)}>
                        <Avatar
                            src={user.avatar}
                            alt={user.name}
                            size="sm"
                        />
                        <span>{user.name}</span>
                    </li>
                )
            )}}
        </ul>
    );

    // Componente contenedor - maneja lógica y estado
    const UserListContainer: React.FC = () => {
        const [users, setUsers] = useState<User[]>([]);
        const [selectedUser, setSelectedUser] = useState<User | null>(null);
        const [loading, setLoading] = useState(true);

        useEffect(() => {
            fetchUsers()
                .then(setUsers)
                .finally(() => setLoading(false));
        }, []);

        if (loading) return <Spinner />;

        return (
            <UserListView
                users={users}
                onSelectUser={setSelectedUser}
                selectedId={selectedUser?.id}
            />
        );
    };
};

```

3.2 Especialización

Crear versiones especializadas de componentes genéricos:

```

// Componente genérico
interface DialogProps {
    title: string;
    children: React.ReactNode;
    onClose: () => void;
    actions?: React.ReactNode;
}

const Dialog: React.FC<DialogProps> = ({
    title,

```

```

    children,
    onClose,
    actions,
  }) => (
    <div
      className="dialog-overlay"
      onClick={onClose}>
      <div
        className="dialog"
        onClick={(e) => e.stopPropagation()}>
        <header className="dialog-header">
          <h2>{title}</h2>
          <button
            onClick={onClose}
            aria-label="Cerrar">
            x
          </button>
        </header>
        <div className="dialog-body">{children}</div>
        {actions} && <footer className="dialog-actions">{actions}</footer>
      </div>
    </div>
  );

```

// Especialización: Diálogo de confirmación

```

interface ConfirmDialogProps {
  title: string;
  message: string;
  onConfirm: () => void;
  onCancel: () => void;
  confirmText?: string;
  cancelText?: string;
}

```

```

const ConfirmDialog: React.FC<ConfirmDialogProps> = ({
  title,
  message,
  onConfirm,
  onCancel,
  confirmText = 'Confirmar',
  cancelText = 'Cancelar',
}) => (
  <Dialog
    title={title}
    onClose={onCancel}
    actions={
      <>
        <button
          onClick={onCancel}
          className="btn-secondary">
            {cancelText}
          </button>
        <button
          onClick={onConfirm}
          className="btn-primary">
            {confirmText}

```

```

        </button>
      </>
    }>
    <p>{message}</p>
  </Dialog>
);

// Especialización: Diálogo de error
const ErrorDialog: React.FC<{ error: string; onClose: () => void }> = ({
  error,
  onClose,
}) => (
  <Dialog
    title="Error"
    onClose={onClose}
    actions={
      <button
        onClick={onClose}
        className="btn-primary">
        Entendido
      </button>
    }>
    <div className="error-content">
      <span className="error-icon">⚠️</span>
      <p>{error}</p>
    </div>
  </Dialog>
);

```

3.3 Inversión de Control

Dejar que el padre controle el comportamiento:

```

// Componente con inversión de control
interface ListProps<T> {
  items: T[];
  renderItem: (item: T, index: number) => React.ReactNode;
  keyExtractor: (item: T) => string;
  emptyMessage?: string;
}

function List<T>({
  items,
  renderItem,
  keyExtractor,
  emptyMessage,
}: ListProps<T>) {
  if (items.length === 0) {
    return <p className="empty-list">{emptyMessage ?? 'No hay elementos'}</p>;
  }

  return (
    <ul className="list">
      {items.map((item, index) => (
        <li key={keyExtractor(item)}>{renderItem(item, index)}</li>
      ))}
    </ul>
  )
}

```

```

    </ul>
  );
}

// Uso - el padre controla cómo se renderiza cada item
const UserDirectory: React.FC = () => {
  const users: User[] = [
    /* ... */
  ];

  return (
    <List
      items={users}
      keyExtractor={(user) => user.id}
      renderItem={(user) => (
        <div className="user-row">
          <Avatar
            src={user.avatar}
            alt={user.name}
          />
          <span>{user.name}</span>
          <span className="email">{user.email}</span>
        </div>
      )}
      emptyMessage="No hay usuarios registrados"
    />
  );
};

const ProductCatalog: React.FC = () => {
  const products: Product[] = [
    /* ... */
  ];

  return (
    <List
      items={products}
      keyExtractor={(product) => product.sku}
      renderItem={(product) => (
        <div className="product-item">
          <img
            src={product.image}
            alt={product.name}
          />
          <h3>{product.name}</h3>
          <span className="price">${product.price}</span>
        </div>
      )}
    />
  );
};

```

4. Higher-Order Components (HOCs)

Los HOCs son funciones que reciben un componente y retornan uno nuevo con funcionalidad adicional:

```

// HOC para añadir loading state
function withLoading<P extends object>({
  WrappedComponent: React.ComponentType<P>,
  loadingMessage: string = 'Cargando...',
}) {
  interface WithLoadingProps {
    isLoading: boolean;
  }

  return function WithLoadingComponent(props: P & WithLoadingProps) {
    const { isLoading, ...componentProps } = props;

    if (isLoading) {
      return <div className="loading">{loadingMessage}</div>;
    }

    return <WrappedComponent {...(componentProps as P)} />;
  };
}

// Uso
const UserList: React.FC<{ users: User[] }> = ({ users }) => (
  <ul>
    {users.map((u) => (
      <li key={u.id}>{u.name}</li>
    ))}
  </ul>
);

const UserListWithLoading = withLoading(UserList, 'Cargando usuarios...');

// En otro componente
const App: React.FC = () => {
  const [users, setUsers] = useState<User[]>([]);
  const [loading, setLoading] = useState(true);

  return (
    <UserListWithLoading
      users={users}
      isLoading={loading}
    />
  );
};

```

HOC con TypeScript - Mejor tipado

```

// HOC para añadir autenticación
interface WithAuthProps {
  isAuthenticated: boolean;
  user: User | null;
}

function withAuth<P extends object>({WrappedComponent: React.ComponentType<P>}) {
  return function WithAuthComponent(props: Omit<P, keyof WithAuthProps>) {
    // Simular contexto de autenticación

```

```

const auth = useAuth();

if (!auth.isAuthenticated) {
  return <Navigate to="/login" />;
}

return (
  <WrappedComponent
    {...(props as P)}
    isAuthenticated={auth.isAuthenticated}
    user={auth.user}
  />
);
};
}

// Componente que requiere autenticación
interface DashboardProps extends WithAuthProps {
  title: string;
}

const Dashboard: React.FC<DashboardProps> = ({ title, user }) => (
  <div>
    <h1>{title}</h1>
    <p>Bienvenido, {user?.name}</p>
  </div>
);

const ProtectedDashboard = withAuth(Dashboard);

// Uso
<ProtectedDashboard title="Mi Dashboard" />;

```

⚠ **Nota:** Los HOCs fueron populares antes de los hooks. Hoy en día, los **custom hooks** suelen ser preferibles por su simplicidad y mejor composición.

5. Composición vs Herencia - Comparativa

Aspecto	Composición	Herencia
Flexibilidad	Alta - combina componentes libremente	Baja - jerarquía fija
Acoplamiento	Bajo - componentes independientes	Alto - depende de la clase base
Reutilización	Fácil - mezcla y combina	Difícil - hereda todo o nada
Testing	Simple - testea cada pieza	Complejo - necesita mocks de la base
En React	✅ Recomendado	❌ Evitar

6. Cuándo Usar Cada Patrón

```

// ✅ Composición simple con children
// Cuando: el contenido interno es completamente variable
const Card: React.FC<{ children: React.ReactNode }> = ({ children }) => (
  <div className="card">{children}</div>

```

```

);

// ✅ Especialización
// Cuando: necesitas versiones pre-configuradas
const SuccessCard = ({ message }: { message: string }) => (
  <Card>
    <span className="icon">✓</span>
    <p>{message}</p>
  </Card>
);

// ✅ Render props / Inversión de control
// Cuando: el padre necesita controlar el renderizado
const DataFetcher = <T,>({
  url,
  render,
}): {
  url: string;
  render: (data: T) => React.ReactNode;
}) => {
  const [data, setData] = useState<T | null>(null);
  useEffect(() => {
    fetch(url)
      .then((r) => r.json())
      .then(setData);
  }, [url]);
  return data ? render(data) : <p>Cargando...</p>;
};

// ✅ HOC
// Cuando: necesitas añadir funcionalidad cross-cutting a múltiples componentes
// (autenticación, logging, analytics)
const withAnalytics = (Component: React.ComponentType) => {
  return (props: any) => {
    useEffect(() => {
      trackPageView(Component.name);
    }, []);
    return <Component {...props} />;
  };
};

```

✅ Checklist de Verificación

- ☐ Entiendo por qué React prefiere composición sobre herencia
- ☐ Puedo crear componentes genéricos que aceptan children
- ☐ Sé cuándo usar especialización vs render props
- ☐ Entiendo los HOCs y cuándo podrían ser útiles
- ☐ Puedo separar componentes de presentación y contenedor

🔗 Recursos Adicionales

- [React Docs - Composition vs Inheritance](#)
- [Patterns.dev - React Patterns](#)
- [Kent C. Dodds - Inversion of Control](#)

Siguiente: [Children y Slots](#)